

SignalMDM

Phase 2 - Mapping & Standardization

Comprehensive End-to-End Developer Implementation Specification

Attribute	Details
Platform	SignalMDM - AI-Native Master Data Intelligence Platform
Phase	Phase 2 - Mapping & Standardization
Primary Goal	Convert Phase 1 staging records into canonical, standardized and normalized records ready for Data Quality validation.
Target Audience	Frontend developers, backend developers, database engineers, data engineers, QA, DevOps and project manager
Prerequisite	Phase 1 Foundation completed: source systems, ingestion runs, raw records, staging entities and READY_FOR_MAPPING records.
Important Developer Note	Phase 2 must not perform Data Quality scoring, matching, entity resolution or golden record creation. It only creates canonical metadata, mappings, transformations, standardized payloads, normalization runs and mapping/normalization error handling.

1. Executive Summary

- Phase 2 establishes the mapping and standardization layer for SignalMDM.
- It consumes staging records created in Phase 1 with staging_status = READY_FOR_MAPPING.
- It allows data architects and data engineers to define canonical entity models, configure source-to-canonical field mappings, apply transformation rules, apply standardization rules and produce normalized records.
- The output of Phase 2 is not only a set of UI screens. It is a controlled canonicalization engine that converts inconsistent source data into master-data-ready records.
- Phase 3 Data Quality will consume Phase 2 records with staging_status = READY_FOR_DQ.

Phase 2 Output: Normalized canonical payloads stored against staging records, normalization run summaries, and mapping/normalization errors captured for remediation. Final eligible status: READY_FOR_DQ.

2. Phase 2 Scope and Boundaries

In Scope	Out of Scope for Phase 2
Canonical model management	Data Quality rule scoring and DQ dashboards
Source-to-canonical field mapping	Matching, entity resolution and clustering
Transformation rule configuration and execution	Golden record creation and survivorship
Standardization rule configuration and execution	Stewardship workflows and approvals
Normalization run execution and tracking	External enrichment and signal intelligence
Normalized record monitoring	Downstream publishing or SignalIQ integration
Mapping and normalization error handling	Manual DQ remediation workflows
Seed data for canonical fields, rules and reference values	Production-grade data lake or raw document processing

3. Implementation Principles

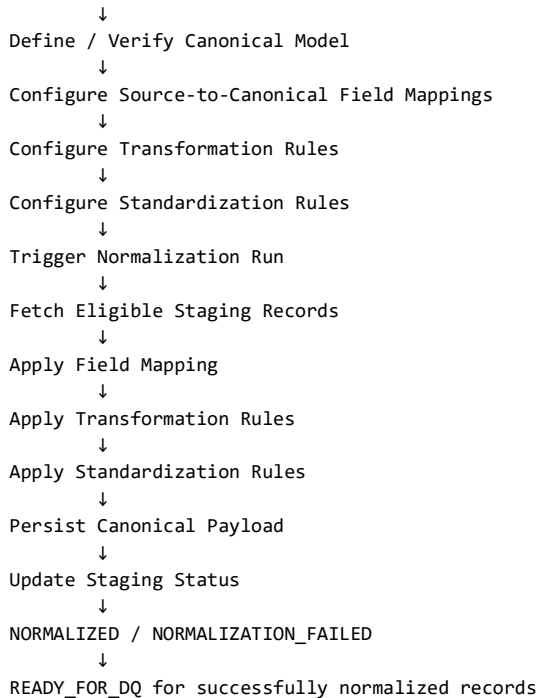
- Canonical-first design: every source field must map into a controlled canonical field before later phases can trust it.
- Metadata-driven mapping: mappings, transformations and standardization rules must be database-driven and configurable through UI.
- Backend is the source of truth: frontend validation improves experience, but all business validation must exist in backend services.
- No raw data mutation: raw_records and raw payload snapshots from Phase 1 must remain unchanged.
- Idempotent normalization: re-running normalization for the same ingestion run must not create duplicate normalized outputs.
- Audit everything: create, update, deactivate and retry actions must be captured in audit logs.
- Clear statuses: staging entities must move through controlled statuses only.
- Separation of concerns: do not mix DQ validation, matching or golden record logic into Phase 2.

4. Phase 2 Functional Modules

Module	Purpose	Key Data Captured / Processed
Canonical Model Management	Define enterprise-standard fields for each entity type.	Entity type, canonical field name, data type, required flag, validation type, standardization type, display name, status.
Field Mapping Engine	Map source fields from Phase 1 staging payloads to canonical fields.	Source system, entity type, source field, canonical field, default value, required flag, mapping status.
Transformation Rules	Define reusable transformation logic before standardization.	Trim, case conversion, regex cleanup, remove special characters, normalize phone/date/country.
Standardization Rules	Define enterprise standard values and formats.	Reference mappings, key-value replacements, regex replacements, case formatting, date and country normalization.
Normalization Execution	Run mapping + transformation + standardization over eligible staging records.	Normalization run, processed count, normalized count, failed count, status, errors.
Normalized Records Monitoring	View raw, mapped and standardized payloads.	Canonical payload JSON, normalized_at, status, record comparison.
Mapping Error Handling	Capture and retry mapping/normalization failures.	Error type, error message, source record, entity type, failed step, retry status.

5. End-to-End Phase 2 Flow

Phase 1 Output: staging_entities where staging_status = READY_FOR_MAPPING



6. User Flow

1. Data architect opens the Canonical Models screen and defines canonical fields for entity types such as CUSTOMER, SUPPLIER, PRODUCT and ACCOUNT.
2. Data engineer opens Field Mappings and selects source system + entity type.
3. System displays source fields and sample values from staging records.
4. Data engineer maps source fields to canonical fields.
5. Data engineer assigns transformation rules where required.
6. Data architect configures standardization rules for values such as country, state, phone, date and codes.
7. User triggers a normalization run for an ingestion run / source system / entity type.
8. Backend creates normalization_run and processes eligible staging records asynchronously.
9. System stores canonical payloads and updates staging records.
10. User monitors Normalization Runs and reviews normalized records or mapping errors.
11. Successful records are marked READY_FOR_DQ for Phase 3.

7. Backend Architecture for Phase 2

```
backend/
  app/
    modules/
      mapping_standardization/
        models.py           # SQLAlchemy models for canonical fields, mappings, rules, runs and errors
        schemas.py          # Pydantic request/response schemas
        routes.py           # API endpoints
        services.py          # Business orchestration
        repositories.py      # Persistence layer
        validators.py        # Mapping, canonical and rule validation
        normalization_engine.py # Core mapping + transform + standardize engine
        transformation_engine.py
        standardization_engine.py
        seed.py              # Initial canonical model and rule seed data
        constants.py         # Enums and status values
      tests/
```

SignalMDM Phase 2 Mapping & Standardization - Comprehensive Developer Specification (private & confidential)

```
test_canonical_models_api.py
test_field_mappings_api.py
test_normalization_engine.py
test_mapping_errors.py
```

Service	Responsibilities
CanonicalModelService	Create/update/list canonical fields, validate uniqueness, ensure field names are snake_case, expose fields by entity type.
FieldMappingService	Create/update/list source-to-canonical mappings, validate canonical field existence, prevent duplicate mappings.
TransformationRuleService	Manage transformation rules and execute transformation chains.
StandardizationRuleService	Manage standardization rules and execute standardization logic.
NormalizationService	Create normalization runs, select eligible staging records, orchestrate mapping, transformation and standardization.
MappingErrorService	Persist mapping/normalization errors, support retry, expose errors to UI.
AuditService	Capture create/update/deactivate/retry events for canonical models, mappings and runs.

8. Frontend Architecture for Phase 2

```
frontend/src/
  pages/
    canonicalModels/
    fieldMappings/
    transformationRules/
    standardizationRules/
    normalizationRuns/
    normalizedRecords/
    mappingErrors/
  components/
    mapping/
    transformations/
    jsonViewer/
    tables/
    comparison/
    statusBadges/
  services/
    canonical/
    mappings/
    transformations/
    standardization/
    normalization/
  hooks/
    useCanonicalModels.js
    useFieldMappings.js
    useNormalizationRuns.js
    useMappingErrors.js
```

9. Core Data Model Overview

Entity / Table	Why It Exists	Primary Relationships
canonical_fields	Defines the standard enterprise schema per entity type.	Used by field_mappings and normalization engine.
field_mappings	Maps source fields into canonical fields.	Links source_systems, entity_types and canonical_fields.
transformation_rules	Stores reusable transformation logic.	Referenced by field_mappings or normalization profiles.
standardization_rules	Stores reusable standardization configurations.	Applied after transformations.
normalization_runs	Tracks execution of normalization batches.	Linked to ingestion_runs, source_systems and staging_entities.
mapping_errors	Captures mapping/transformation/standardization failures.	Linked to staging_entities and normalization_runs.
staging_entities updates	Stores canonical_payload_json and Phase 2 statuses.	Consumes Phase 1 staging records and prepares Phase 3 input.

10. Database Specification

10.1 canonical_fields

Column	Type	Constraint	Description
id	UUID	PK	System-generated canonical field ID.
tenant_id	UUID	FK tenants.id, not null	Tenant scope.
entity_type	VARCHAR(100)	not null	CUSTOMER, SUPPLIER, PRODUCT, ACCOUNT, ASSET, LOCATION.
canonical_field_name	VARCHAR(255)	not null	Technical canonical field name, e.g. customer_name.
display_name	VARCHAR(255)	nullable	User-friendly label.
description	TEXT	nullable	Business meaning of the field.
data_type	VARCHAR(50)	not null	TEXT, NUMBER, DATE, BOOLEAN, JSON, EMAIL, PHONE.
field_length	INT	nullable	Optional max length.
is_required	BOOLEAN	default false	Used later by DQ and UI.
validation_type	VARCHAR(100)	nullable	EMAIL, PHONE, COUNTRY, DATE, NUMERIC, TEXT, URL, GST.
standardization_type	VARCHAR(100)	nullable	NAME, EMAIL, PHONE, COUNTRY, STATE, CITY, CODE, DATE, TEXT.
status	VARCHAR(50)	default ACTIVE	ACTIVE, INACTIVE, ARCHIVED.
created_by	VARCHAR(255)	nullable	User who created the field.
created_at	TIMESTAMP	not null	Created timestamp.
updated_at	TIMESTAMP	not null	Updated timestamp.

```
CREATE TABLE canonical_fields (  
  id UUID PRIMARY KEY,  
  tenant_id UUID NOT NULL REFERENCES tenants(id),  
  entity_type VARCHAR(100) NOT NULL,  
  canonical_field_name VARCHAR(255) NOT NULL,  
  display_name VARCHAR(255),  
  description TEXT,  
  data_type VARCHAR(50) NOT NULL,  
  field_length INT,  
  is_required BOOLEAN DEFAULT FALSE,  
  validation_type VARCHAR(100),  
  standardization_type VARCHAR(100),  
  status VARCHAR(50) DEFAULT 'ACTIVE',  
  created_by VARCHAR(255),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE (tenant_id, entity_type, canonical_field_name)  
);
```

10.2 field_mappings

Column	Type	Constraint	Description
id	UUID	PK	Mapping identifier.
tenant_id	UUID	FK tenants.id	Tenant scope.
source_system_id	UUID	FK source_systems.id	Source system from Phase 1.
entity_type	VARCHAR(100)	not null	Entity type of mapping.
source_field_name	VARCHAR(255)	not null	Field name in source/raw payload.
canonical_field_id	UUID	FK canonical_fields.id	Target canonical field.
canonical_field_name	VARCHAR(255)	not null	Denormalized for easier queries.
transformation_rule_ids	JSONB	default []	Ordered list of transformation rules.
default_value	TEXT	nullable	Fallback when source field is missing or blank.
is_required	BOOLEAN	default false	Whether mapping is mandatory.
mapping_status	VARCHAR(50)	default ACTIVE	ACTIVE, INACTIVE, DRAFT.
created_by	VARCHAR(255)	nullable	User who created mapping.
created_at	TIMESTAMP	not null	Created timestamp.
updated_at	TIMESTAMP	not null	Updated timestamp.

```
CREATE TABLE field_mappings (  
  id UUID PRIMARY KEY,  
  tenant_id UUID NOT NULL REFERENCES tenants(id),  
  source_system_id UUID NOT NULL REFERENCES source_systems(id),  
  entity_type VARCHAR(100) NOT NULL,  
  source_field_name VARCHAR(255) NOT NULL,  
  canonical_field_id UUID REFERENCES canonical_fields(id),  
  canonical_field_name VARCHAR(255) NOT NULL,  
  SignalMDM Phase 2 Mapping & Standardization - Comprehensive Developer Specification (private & confidential)
```

```

transformation_rule_ids JSONB DEFAULT '[]::jsonb,
default_value TEXT,
is_required BOOLEAN DEFAULT FALSE,
mapping_status VARCHAR(50) DEFAULT 'ACTIVE',
created_by VARCHAR(255),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
UNIQUE (tenant_id, source_system_id, entity_type, source_field_name)
);

```

10.3 transformation_rules

Column	Type	Constraint	Description
id	UUID	PK	Transformation rule ID.
tenant_id	UUID	FK tenants.id	Tenant scope.
rule_code	VARCHAR(150)	unique per tenant	Technical code.
rule_name	VARCHAR(255)	not null	Rule display name.
transformation_type	VARCHAR(100)	not null	TRIM, UPPERCASE, LOWERCASE, REMOVE_SPECIAL_CHARS, REGEX_REPLACE, NORMALIZE_PHONE, NORMALIZE_DATE.
rule_config	JSONB	not null	Config for transformation engine.
status	VARCHAR(50)	default ACTIVE	ACTIVE / INACTIVE.
created_by	VARCHAR(255)	nullable	User who created rule.
created_at	TIMESTAMP	not null	Created timestamp.
updated_at	TIMESTAMP	not null	Updated timestamp.

```

CREATE TABLE transformation_rules (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  rule_code VARCHAR(150) NOT NULL,
  rule_name VARCHAR(255) NOT NULL,
  transformation_type VARCHAR(100) NOT NULL,
  rule_config JSONB NOT NULL,
  status VARCHAR(50) DEFAULT 'ACTIVE',
  created_by VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (tenant_id, rule_code)
);

```

10.4 standardization_rules

Column	Type	Constraint	Description
id	UUID	PK	Standardization rule ID.
tenant_id	UUID	FK tenants.id	Tenant scope.
rule_code	VARCHAR(150)	unique per tenant	Technical code.
standardization_type	VARCHAR(100)	not null	COUNTRY, STATE, CITY, PHONE, DATE, CODE, NAME, TEXT.
rule_name	VARCHAR(255)	not null	Display name.
rule_config	JSONB	not null	Key-value mappings, regex, case normalization.
status	VARCHAR(50)	default ACTIVE	ACTIVE / INACTIVE.
created_by	VARCHAR(255)	nullable	User who created rule.
created_at	TIMESTAMP	not null	Created timestamp.
updated_at	TIMESTAMP	not null	Updated timestamp.

```

CREATE TABLE standardization_rules (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  rule_code VARCHAR(150) NOT NULL,
  standardization_type VARCHAR(100) NOT NULL,
  rule_name VARCHAR(255) NOT NULL,
  rule_config JSONB NOT NULL,
  status VARCHAR(50) DEFAULT 'ACTIVE',
  created_by VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (tenant_id, rule_code)
);

```

10.5 normalization_runs

Column	Type	Constraint	Description
id	UUID	PK	Normalization run ID.
tenant_id	UUID	FK tenants.id	Tenant scope.
ingestion_run_id	UUID	FK ingestion_runs.id	Source ingestion run.
source_system_id	UUID	FK source_systems.id	Source system.
entity_type	VARCHAR(100)	not null	Entity type processed.
status	VARCHAR(50)	default CREATED	CREATED, RUNNING, COMPLETED, FAILED, PARTIAL_SUCCESS.
total_records	INT	default 0	Eligible records count.
normalized_records	INT	default 0	Successfully normalized.
failed_records	INT	default 0	Failed records.
error_summary	JSONB	nullable	Aggregated error summary.
started_at	TIMESTAMP	nullable	Start time.
completed_at	TIMESTAMP	nullable	Completion time.
created_by	VARCHAR(255)	nullable	User who triggered run.
created_at	TIMESTAMP	not null	Created timestamp.

```
CREATE TABLE normalization_runs (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  ingestion_run_id UUID NOT NULL REFERENCES ingestion_runs(id),
  source_system_id UUID NOT NULL REFERENCES source_systems(id),
  entity_type VARCHAR(100) NOT NULL,
  status VARCHAR(50) DEFAULT 'CREATED',
  total_records INT DEFAULT 0,
  normalized_records INT DEFAULT 0,
  failed_records INT DEFAULT 0,
  error_summary JSONB,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  created_by VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

10.6 mapping_errors

Column	Type	Constraint	Description
id	UUID	PK	Error identifier.
tenant_id	UUID	FK tenants.id	Tenant scope.
normalization_run_id	UUID	FK normalization_runs.id	Related normalization run.
staging_entity_id	UUID	FK staging_entities.id	Failed staging record.
source_system_id	UUID	FK source_systems.id	Source system.
entity_type	VARCHAR(100)	not null	Entity type.
error_type	VARCHAR(100)	not null	MISSING_MAPPING, TRANSFORMATION_FAILED, STANDARDIZATION_FAILED, PAYLOAD_ERROR.
error_message	TEXT	not null	Human-readable error.
source_field_name	VARCHAR(255)	nullable	Failed source field.
canonical_field_name	VARCHAR(255)	nullable	Target canonical field if applicable.
payload_snapshot_json	JSONB	nullable	Record payload at failure time.
status	VARCHAR(50)	default OPEN	OPEN, RETRIED, RESOLVED, IGNORED.
created_at	TIMESTAMP	not null	Created timestamp.
resolved_at	TIMESTAMP	nullable	Resolved timestamp.

```
CREATE TABLE mapping_errors (
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES tenants(id),
  normalization_run_id UUID REFERENCES normalization_runs(id),
  staging_entity_id UUID REFERENCES staging_entities(id),
  source_system_id UUID REFERENCES source_systems(id),
  entity_type VARCHAR(100) NOT NULL,
  error_type VARCHAR(100) NOT NULL,
  error_message TEXT NOT NULL,
  source_field_name VARCHAR(255),
  canonical_field_name VARCHAR(255),
  payload_snapshot_json JSONB,
  status VARCHAR(50) DEFAULT 'OPEN',

```

```

        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        resolved_at TIMESTAMP
    );

```

10.7 staging_entities Phase 2 Columns

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS normalization_run_id UUID REFERENCES normalization_runs(id);

```

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS mapped_payload_json JSONB;

```

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS standardized_payload_json JSONB;

```

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS normalization_status VARCHAR(50) DEFAULT 'NOT_STARTED';

```

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS normalized_at TIMESTAMP;

```

```

ALTER TABLE staging_entities
ADD COLUMN IF NOT EXISTS normalization_error TEXT;

```

11. Required Indexes

```

CREATE INDEX idx_canonical_fields_entity
ON canonical_fields (tenant_id, entity_type, status);

```

```

CREATE INDEX idx_field_mappings_source_entity
ON field_mappings (tenant_id, source_system_id, entity_type, mapping_status);

```

```

CREATE INDEX idx_normalization_runs_ingestion
ON normalization_runs (tenant_id, ingestion_run_id, source_system_id, entity_type);

```

```

CREATE INDEX idx_mapping_errors_run
ON mapping_errors (tenant_id, normalization_run_id, status);

```

```

CREATE INDEX idx_staging_phase2_status
ON staging_entities (tenant_id, ingestion_run_id, source_system_id, entity_type, staging_status);

```

12. Enumerations and Reference Values

Enum	Allowed Values
Entity Type	CUSTOMER, SUPPLIER, PRODUCT, ACCOUNT, ASSET, LOCATION, CONTACT
Canonical Field Status	ACTIVE, INACTIVE, ARCHIVED
Mapping Status	DRAFT, ACTIVE, INACTIVE
Transformation Type	TRIM, UPPERCASE, LOWERCASE, TITLE_CASE, REMOVE_SPECIAL_CHARS, REGEX_REPLACE, NORMALIZE_PHONE, NORMALIZE_DATE, NORMALIZE_COUNTRY
Standardization Type	NAME, EMAIL, PHONE, ADDRESS, COUNTRY, STATE, CITY, DATE, CODE, TEXT
Normalization Run Status	CREATED, RUNNING, COMPLETED, FAILED, PARTIAL_SUCCESS, CANCELLED
Staging Status	READY_FOR_MAPPING, MAPPING_IN_PROGRESS, NORMALIZED, NORMALIZATION_FAILED, READY_FOR_DQ
Mapping Error Type	MISSING_MAPPING, INVALID_CANONICAL_FIELD, TRANSFORMATION_FAILED, STANDARDIZATION_FAILED, PAYLOAD_ERROR, DUPLICATE_MAPPING
Mapping Error Status	OPEN, RETRIED, RESOLVED, IGNORED

13. Required Initial Data / Seed Data

Seed Area	Example Records / Values	Purpose
Entity Types	CUSTOMER, SUPPLIER, PRODUCT, ACCOUNT, ASSET, LOCATION	Used across canonical models and mapping screens.
Customer Canonical Fields	customer_name, customer_legal_name, primary_email, primary_phone, website, gst_number, country, state, city, postal_code, industry, status	Initial CUSTOMER model for implementation and testing.

Supplier Canonical Fields	supplier_name, supplier_legal_name, primary_email, primary_phone, gst_number, country, state, city, supplier_category, status	Initial SUPPLIER model.
Product Canonical Fields	product_code, product_name, product_family, product_category, unit_of_measure, status	Initial PRODUCT model.
Transformation Rules	TRIM_SPACES, LOWERCASE_EMAIL, REMOVE_SPECIAL_CHARS, NORMALIZE_PHONE, NORMALIZE_DATE	Reusable transformation rules.
Standardization Rules	COUNTRY_STANDARDIZATION, STATE_STANDARDIZATION, CUSTOMER_STATUS_STANDARDIZATION	Reference conversions.
Sample Source Mappings	customerName -> customer_name; emailId -> primary_email; mobileNo -> primary_phone; country -> country	Test Phase 2 with Phase 1 staging data.

13.1 Sample Seed JSON - Customer Canonical Model

```
[
  {
    "entity_type": "CUSTOMER",
    "canonical_field_name": "customer_name",
    "data_type": "TEXT",
    "is_required": true,
    "validation_type": "TEXT",
    "standardization_type": "NAME"
  },
  {
    "entity_type": "CUSTOMER",
    "canonical_field_name": "primary_email",
    "data_type": "EMAIL",
    "is_required": false,
    "validation_type": "EMAIL",
    "standardization_type": "EMAIL"
  },
  {
    "entity_type": "CUSTOMER",
    "canonical_field_name": "primary_phone",
    "data_type": "PHONE",
    "is_required": false,
    "validation_type": "PHONE",
    "standardization_type": "PHONE"
  },
  {
    "entity_type": "CUSTOMER",
    "canonical_field_name": "country",
    "data_type": "TEXT",
    "is_required": false,
    "validation_type": "COUNTRY",
    "standardization_type": "COUNTRY"
  }
]
```

14. API Design Standards

Standard success response:

```
{
  "status": "success",
  "message": "Operation completed successfully",
  "data": { ... },
  "meta": {
    "request_id": "REQ-...",
    "timestamp": "2026-05-25T10:00:00Z"
  }
}
```

Standard error response:

```
{
  "status": "error",
  "message": "Validation failed",
  "errors": [
    {
      "field": "canonical_field_name",
      "message": "Field name must be snake_case"
    }
  ],
  "meta": {
    "request_id": "REQ-...",
    "timestamp": "2026-05-25T10:00:00Z"
  }
}
```

Method	Endpoint	Purpose	Roles
GET	/api/mdm/canonical-models	List canonical fields with filters/pagination	Admin, Data Engineer, Data Steward, Viewer
POST	/api/mdm/canonical-models	Create canonical field	Admin, Data Engineer
PUT	/api/mdm/canonical-models/{id}	Update canonical field	Admin, Data Engineer
PATCH	/api/mdm/canonical-models/{id}/status	Activate/deactivate canonical field	Admin
GET	/api/mdm/field-mappings	List mappings by source/entity	Admin, Data Engineer, Viewer
POST	/api/mdm/field-mappings	Create mapping	Admin, Data Engineer
PUT	/api/mdm/field-mappings/{id}	Update mapping	Admin, Data Engineer
DELETE	/api/mdm/field-mappings/{id}	Soft deactivate mapping	Admin
GET	/api/mdm/transformation-rules	List transformation rules	Admin, Data Engineer, Viewer
POST	/api/mdm/transformation-rules	Create transformation rule	Admin, Data Engineer
GET	/api/mdm/standardization-rules	List standardization rules	Admin, Data Engineer, Viewer
POST	/api/mdm/standardization-rules	Create standardization rule	Admin, Data Engineer
POST	/api/mdm/normalization-runs	Trigger normalization run	Admin, Data Engineer
GET	/api/mdm/normalization-runs	List normalization runs	Admin, Data Engineer, Viewer

GET	/api/mdm/normalization-runs/{id}/status	Get run status	Admin, Data Engineer, Viewer
GET	/api/mdm/normalized-records	List normalized records	Admin, Data Engineer, Data Steward, Viewer
GET	/api/mdm/normalized-records/{staging_entity_id}	View record comparison	Admin, Data Engineer, Data Steward, Viewer
GET	/api/mdm/mapping-errors	List mapping errors	Admin, Data Engineer, Data Steward
POST	/api/mdm/mapping-errors/{id}/retry	Retry failed record after correction	Admin, Data Engineer

15. Detailed API Contracts

15.1 Create Canonical Field

POST /api/mdm/canonical-models

Request:

```
{
  "tenant_id": "uuid",
  "entity_type": "CUSTOMER",
  "canonical_field_name": "customer_name",
  "display_name": "Customer Name",
  "description": "Primary display name of customer",
  "data_type": "TEXT",
  "field_length": 255,
  "is_required": true,
  "validation_type": "TEXT",
  "standardization_type": "NAME",
  "status": "ACTIVE"
}
```

Response data:

```
{
  "id": "uuid",
  "entity_type": "CUSTOMER",
  "canonical_field_name": "customer_name",
  "status": "ACTIVE",
  "created_at": "timestamp"
}
```

15.2 Create Field Mapping

POST /api/mdm/field-mappings

Request:

```
{
  "tenant_id": "uuid",
  "source_system_id": "uuid",
  "entity_type": "CUSTOMER",
  "source_field_name": "customerName",
  "canonical_field_name": "customer_name",
  "transformation_rule_ids": ["rule_uuid_1", "rule_uuid_2"],
  "default_value": null,
  "is_required": true,
  "mapping_status": "ACTIVE"
}
```

Validation:

- Source system must exist and be active.
- Canonical field must exist and be active for entity_type.
- Duplicate active mapping for source_system + entity_type + source_field_name is not allowed.

15.3 Trigger Normalization Run

POST /api/mdm/normalization-runs

Request:

```
{
  "tenant_id": "uuid",
```

```

"ingestion_run_id": "uuid",
"source_system_id": "uuid",
"entity_type": "CUSTOMER",
"created_by": "data.engineer@innovant.ai"
}

```

Response data:

```

{
  "normalization_run_id": "uuid",
  "status": "RUNNING",
  "message": "Normalization started"
}

```

16. Backend Service Logic

Service Function	Logic
create_canonical_field(payload, user_context)	Validate entity_type, snake_case field name, uniqueness, enum values; insert canonical_fields; write audit event.
create_field_mapping(payload, user_context)	Validate source system, canonical field, duplicate mapping and transformation rules; insert mapping; write audit event.
run_normalization(payload, user_context)	Create normalization_run; fetch eligible staging_entities; set status RUNNING; enqueue NORMALIZATION_JOB.
process_record(staging_entity)	Read raw_payload_snapshot_json; apply mappings; apply transformation chain; apply standardization; update payloads/status.
handle_mapping_error(record, error)	Persist mapping_errors row; update staging_entities.normalization_status and staging_status to NORMALIZATION_FAILED.
retry_mapping_error(error_id)	Validate correction exists; reprocess staging record; update error status to RETRIED or RESOLVED.

17. Normalization Engine Logic

```

def normalize_ingestion_run(normalization_run_id):
    run = get_normalization_run(normalization_run_id)
    records = get_staging_records(
        ingestion_run_id=run.ingestion_run_id,
        source_system_id=run.source_system_id,
        entity_type=run.entity_type,
        staging_status='READY_FOR_MAPPING'
    )

    update_run_status(run.id, 'RUNNING')

    for record in records:
        try:
            update_staging_status(record.id, 'MAPPING_IN_PROGRESS')

            mappings = get_active_mappings(
                source_system_id=record.source_system_id,
                entity_type=record.entity_type
            )

            mapped_payload = {}
            for mapping in mappings:
                value = extract_value(record.raw_payload_snapshot_json, mapping.source_field_name)
                if value is None:
                    value = mapping.default_value

                value = apply_transformation_chain(value, mapping.transformation_rule_ids)
                value = apply_standardization(value, mapping.canonical_field_name)
                mapped_payload[mapping.canonical_field_name] = value

            update_staging_entity(
                record.id,
                mapped_payload_json=mapped_payload,
                standardized_payload_json=mapped_payload,
                canonical_payload_json=mapped_payload,

```

```

        normalization_status='NORMALIZED',
        staging_status='READY_FOR_DQ',
        normalized_at=now()
    )

except Exception as exc:
    create_mapping_error(run.id, record.id, exc)
    update_staging_entity(
        record.id,
        normalization_status='NORMALIZATION_FAILED',
        staging_status='NORMALIZATION_FAILED',
        normalization_error=str(exc)
    )

update_normalization_run_summary(run.id)

```

18. Validation Rules by Module

Module	Validation Rules
Canonical Models	canonical_field_name required, snake_case only, unique per tenant + entity_type, data_type valid, validation_type valid, standardization_type valid.
Field Mappings	source_system_id required, source_field_name required, canonical field exists, duplicate active mapping not allowed, transformation rules active.
Transformation Rules	rule_code unique, transformation_type valid, rule_config must match type. Regex rules must compile.
Standardization Rules	rule_code unique, standardization_type valid, rule_config must be valid JSON. Key-value mappings cannot have duplicate keys.
Normalization Run	ingestion_run_id must exist, source_system_id must match ingestion run, eligible READY_FOR_MAPPING records must exist, active mappings must exist.
Mapping Errors Retry	error must be OPEN, related staging record must exist, mapping/rule correction must be available before retry.

19. Status Transition Rules

Current Status	Allowed Next Status	Notes
READY FOR MAPPING	MAPPING IN PROGRESS	Normalization job has picked the record.
MAPPING IN PROGRESS	NORMALIZED	Mapping + transformation + standardization completed.
MAPPING IN PROGRESS	NORMALIZATION FAILED	Processing failed. Error must be captured.
NORMALIZED	READY_FOR_DQ	Record is eligible for Phase 3 DQ.
NORMALIZATION FAILED	MAPPING IN PROGRESS	Allowed only through retry after correction.

20. Frontend Screen Specifications

Screen	Required Capabilities	Primary Roles
Canonical Models	List canonical fields, filter by entity type/status, create/edit fields, import/export canonical model, inline edit support.	Admin, Data Engineer, Architect
Field Mappings	Filter by source system + entity type, view source sample values, map to canonical fields, select transformation rules, preview before/after.	Data Engineer, Architect
Transformation Rules	Create/edit reusable transformation rules, JSON config editor, validation preview.	Admin, Data Engineer
Standardization Rules	Create key-value, regex, replacement and case normalization rules.	Admin, Data Engineer
Normalization Runs	Trigger normalization, monitor status, counts, timeline and recent errors.	Admin, Data Engineer, Viewer
Normalized Records	View raw payload, mapped payload, standardized payload and canonical payload. Compare before/after.	Data Engineer, Steward, Viewer
Mapping Errors	List failures, view payload, edit mapping, retry failed records.	Data Engineer, Steward

21. Frontend Component Behavior

Component	Behavior
CanonicalModelTable	Reusable table with server-side pagination, filters, sorting, import/export, empty/loading states.
CanonicalFieldModal	Create/edit canonical field with dynamic validation and enum dropdowns.
FieldMappingGrid	Main source-to-canonical mapping grid with inline edit, preview, transformation chips and status badges.
MappingPreviewWidget	Shows source value, transformed value and final standardized value.
RuleConfigEditor	JSON editor with validation and templates for common rules.
NormalizationRunDrawer	Shows execution timeline, counts, failure summary and recent errors.

PayloadComparisonView	Three-column raw vs mapped vs standardized view with copy and JSON expand/collapse.
MappingErrorPanel	Displays error details, payload snapshot and retry actions.

22. Suggested Routes and Navigation

/signalmdm/mapping/canonical-models
 /signalmdm/mapping/field-mappings
 /signalmdm/mapping/transformation-rules
 /signalmdm/mapping/standardization-rules
 /signalmdm/mapping/normalization-runs
 /signalmdm/mapping/normalized-records
 /signalmdm/mapping/mapping-errors

23. Security and RBAC Requirements

Capability	Admin	Data Engineer	Data Steward	Viewer
View canonical models	Yes	Yes	Yes	Yes
Create/edit canonical fields	Yes	Yes	No	No
Create/edit field mappings	Yes	Yes	No	No
Create/edit transformation rules	Yes	Yes	No	No
Trigger normalization run	Yes	Yes	No	No
View normalized records	Yes	Yes	Yes	Yes
Retry mapping errors	Yes	Yes	Limited	No
View audit trail	Yes	Yes	Yes	No

24. Error Handling Requirements

Scenario	HTTP Status	Response Message
Duplicate canonical field	409	Canonical field already exists for entity type.
Invalid canonical field name	422	Field name must be snake_case and contain no spaces.
Invalid enum value	400	Invalid value provided for field.
Source system not found	404	Source system not found.
Canonical field not found	404	Canonical field not found or inactive.
Duplicate mapping	409	Mapping already exists for selected source field.
No eligible records	400	No READY_FOR_MAPPING records found for selected run.
Transformation failure	422	Transformation failed for selected field.
Unauthorized access	401	Authentication required.
Insufficient permission	403	You do not have permission to perform this action.
Unexpected server error	500	Internal server error. Request has been logged.

25. Testing Strategy

Test Category	What to Test	Owner
Unit Tests	Transformation utilities, standardization utilities, mapping validators, normalization service functions.	Backend Developer
API Tests	Canonical model APIs, field mapping APIs, rule APIs, normalization run APIs, mapping error APIs.	Backend Developer / QA
Frontend Tests	Form validation, mapping grid, JSON viewer, run monitor, error retry UI.	Frontend Developer / QA
Integration Tests	Create canonical field -> create mapping -> run normalization -> verify READY_FOR_DQ.	QA / Full-stack
Security Tests	RBAC restrictions, tenant isolation, no unauthorized updates.	QA / DevOps
Regression Tests	Phase 1 source, ingestion, raw and staging screens continue working.	QA

26. Test Cases and Acceptance Criteria

Test ID	Scenario	Expected Result
TC-01	Create CUSTOMER canonical field customer_name	Field created and visible in Canonical Models list.
TC-02	Create duplicate canonical field	API returns 409 and no duplicate is created.

TC-03	Create canonical field with spaces	Validation fails.
TC-04	Create mapping customerName -> customer_name	Mapping created and visible in Field Mappings.
TC-05	Create duplicate mapping for same source field	API returns duplicate mapping error.
TC-06	Preview mapping transformation	Before/after preview displays expected output.
TC-07	Trigger normalization for valid ingestion run	Normalization run created and completes successfully.
TC-08	Record with missing mapping	Mapping error is captured.
TC-09	Successful normalization	staging_entities canonical_payload_json populated and status becomes READY_FOR_DQ.
TC-10	Retry mapping error after correction	Record reprocessed and error marked RESOLVED/RETRIED.
TC-11	Viewer attempts to create mapping	API returns 403.
TC-12	Phase 1 regression	READY_FOR_MAPPING records from Phase 1 remain visible and consumable.

27. Development Sequence for Team Execution

1. Review Phase 1 output and confirm staging_entities records are available with staging_status = READY_FOR_MAPPING.
2. Create Phase 2 backend module folder structure.
3. Create database migrations for canonical_fields, field_mappings, transformation_rules, standardization_rules, normalization_runs and mapping_errors.
4. Add Phase 2 columns to staging_entities.
5. Seed initial canonical fields, transformation rules and standardization rules.
6. Implement SQLAlchemy models and Pydantic schemas.
7. Implement repositories and validators.
8. Implement canonical model APIs.
9. Implement field mapping APIs.
10. Implement transformation and standardization rule APIs.
11. Implement normalization engine and normalization run APIs.
12. Implement mapping error capture and retry APIs.
13. Implement frontend navigation and Phase 2 page skeletons.
14. Implement Canonical Models screen.
15. Implement Field Mappings screen with preview panel.
16. Implement Transformation Rules and Standardization Rules screens.
17. Implement Normalization Runs and Normalized Records screens.
18. Implement Mapping Errors screen and retry action.
19. Perform integration testing from READY_FOR_MAPPING to READY_FOR_DQ.
20. Freeze Phase 2 API contracts before Phase 3 Data Quality implementation begins.

28. Recommended Developer Task Breakdown

Workstream	Tasks	Suggested Owner
Backend - Data Model	Migrations, indexes, seed data, Phase 2 staging column updates.	Rickson / Jefory
Backend - APIs	Canonical models, mappings, transformation rules, standardization rules APIs.	Rickson / Jefory
Backend - Engine	Normalization engine, mapping error handling, retry processing.	Rickson / Jefory
Frontend - Core Screens	Canonical Models, Field Mappings, Transformation Rules screens.	Poorvith
Frontend - Processing Screens	Normalization Runs, Normalized Records, Mapping Errors, comparison views.	Ibrahim
QA	API tests, UI tests, integration tests, regression tests.	QA / Team
PM / Lead	Scope control, blockers, daily tracking, acceptance review.	Jitendra

29. Definition of Done

- All Phase 2 database migrations are committed and executable.
- Initial canonical model seed data exists for CUSTOMER, SUPPLIER and PRODUCT at minimum.
- Canonical model APIs work with standard response and error formats.
- Field mapping APIs work and prevent duplicate active mappings.
- Transformation and standardization rules are configurable from UI and backend.
- Normalization engine processes READY_FOR_MAPPING records and produces canonical_payload_json.
- Successful records are marked READY_FOR_DQ.
- Failures are captured in mapping_errors with actionable messages.

SignalMDM Phase 2 Mapping & Standardization - Comprehensive Developer Specification (private & confidential)

- Frontend Phase 2 screens are connected to real backend APIs.
- Audit logs are written for create/update/deactivate/retry operations.
- RBAC is enforced in backend APIs.
- Swagger/OpenAPI documentation is available.
- QA test cases pass for positive, negative, RBAC and regression scenarios.
- Phase 3 team can consume READY_FOR_DQ records for Data Quality validation.

30. Key Risks and Mitigations

Risk	Impact	Mitigation
Canonical model becomes inconsistent across sources	Later DQ and matching become unreliable	Define canonical fields centrally and enforce uniqueness.
Mappings are hardcoded in backend	UI cannot manage changes and product becomes rigid	Keep mappings metadata-driven in field_mappings table.
Transformation logic is mixed with DQ	Phase boundaries become unclear	Limit Phase 2 to mapping/standardization only.
Too many values kept only in JSONB	Reporting and filters become difficult	Use typed metadata for core objects and JSONB for variable payload only.
No error retry mechanism	Failures block progress	Build mapping_errors and retry APIs from the start.
Frontend hardcodes dropdowns	Backend and UI drift over time	Expose enums/reference data or shared constants.
Phase 1 regression ignored	Foundation breaks while Phase 2 is added	Run Phase 1 regression tests before sign-off.

31. Handover Checklist for Phase 2 Completion

- Database migration scripts committed.
- Seed data script committed and documented.
- Backend APIs deployed in development environment.
- Frontend Phase 2 pages merged and working.
- Swagger/OpenAPI documentation reviewed.
- Postman collection or API test collection available.
- QA test report completed.
- Known issues documented.
- Phase 3 dependency note published: fields and statuses Phase 3 should consume.

32. Phase 2 Final Outcome

Final Outcome: At the end of Phase 2, SignalMDM will have a controlled mapping and standardization layer. Source records from Phase 1 will be converted into canonical, normalized records with clear auditability, error handling and readiness for Phase 3 Data Quality.

```

Phase 1 Output: READY_FOR_MAPPING
      ↓
Phase 2 Processing: Mapping + Transformation + Standardization
      ↓
Phase 2 Output: READY_FOR_DQ
      ↓
Next Phase: Data Quality Validation

```